

Core Modules & API Reference

This document provides a comprehensive developer guide for the custom C++ modules and classes implemented in the QUIC Media System. The system is architecturally split into two primary modules:

1. **Media Processing & App Logic** (`include/app`, `src/app`): Handles media file parsing, packetization, priority-aware stream scheduling, and out-of-order reassembly.
2. **Network & QUIC Transport** (`include/transport`, `src/transport`): Handles the encapsulated MsQuic implementation, managing connections, stream setups, Path MTU constraints, stream priorities, and asynchronous callbacks.

Wire Protocol & Headers

`moq::PacketHeader`

- **File Path:** `include/wire.hpp`
- **Description:** A 16-byte structure prepended to every application-layer fragment transmitted over QUIC.

```
#pragma pack(push, 1)
struct PacketHeader {
    uint32_t nalu_id;           ///< Strictly increasing NALU sequence identifier.
    uint32_t payload_len;      ///< Length of the payload immediately following the header.
    uint16_t flags;            ///< Reserved flags for future extension.
    uint16_t reserved;         ///< Alignment padding or future use.
    uint16_t fragment_idx;     ///< Index of this fragment within the parent NALU.
    uint16_t fragment_count;   ///< Total number of fragments representing the complete parent NALU.
};
#pragma pack(pop)
```

Media Processing & App Logic Modules

1. `moq::app::H265FileParser`

- **File Path:** `include/app/h265_parser.hpp`
- **Description:** Loads a raw H.265 video file into memory and segments it into individual NALUs by searching for start code prefixes (`00 00 01` or `00 00 00 01`).

Helper Structure: `moq::app::NALU`

- `const uint8_t* data` : Pointer to the start code prefix.
- `size_t size` : Size of the NALU in bytes (including the prefix).
- `size_t prefix_len` : Length of the start code prefix (3 or 4 bytes).
- `uint8_t get_nalu_type()` : Returns the 6-bit H.265 NALU type.
- `bool is_important()` : Checks if the NALU represents key stream metadata or synchronization boundaries (VPS type 32, SPS type 33, PPS type 34, IRAP/I-frame types 16–21).

Methods:

- `explicit H265FileParser(const std::string& filepath)`

- Initializes the parser for the target file path.
 - `bool open()`
 - Reads the entire file into memory buffers and parses all boundaries. Returns `true` on success.
 - `const std::vector<Nalu>& get_nalus() const`
 - Returns a reference to the parsed list of NALU descriptors.
-

2. `moq::app::DefaultPacketizer`

- **File Path:** `include/app/packetizer.hpp`
- **Description:** Implements the `moq::app::IPacketizer` interface to slice large NALUs (e.g., I-frames exceeding the Path MTU) into smaller packets with a 16-byte packed `PacketHeader`.

Helper Structure: `moq::app::Packet`

- `moq::PacketHeader hdr` : Pre-populated packet header.
- `std::vector<uint8_t> payload` : Payload chunk data.

Methods:

- `explicit DefaultPacketizer(size_t max_payload_size)`
 - Constructs a packetizer enforcing a maximum size boundary (in bytes) on each fragment's payload.
 - `std::vector<Packet> packetize(const Nalu& nalu, uint32_t nalu_id) override`
 - Slices the raw Nalu buffer, prepends sequence headers, and returns a list of fragments.
-

3. `moq::app::Reassembler`

- **File Path:** `include/app/reassembler.hpp`
- **Description:** Used on the server to reorder out-of-order packet fragments arriving on multiple QUIC streams, reassembling them back into contiguous NALUs before writing them to the final output sink.

Types & Callbacks:

- `using WriteFn = std::function<void(const uint8_t* data, size_t len)>`
 - Callback signature invoked when a complete contiguous NALU is successfully reassembled in order.

Methods:

- `explicit Reassembler(WriteFn writer)`
 - Constructs a reassembler with the designated writer callback.
 - `void on_packet(const PacketHeader& hdr, const uint8_t* payload, size_t len)`
 - Processes an incoming packet fragment. It caches out-of-order fragments using a sorted map and flushes sequential completed NALUs.
 - `uint32_t expected_id() const`
 - Returns the next sequential NALU ID expected.
 - `size_t buffered_count() const`
 - Returns the number of partially received or out-of-order NALUs currently occupying the buffer map.
-

4. `moq::app::FileSink`

- **File Path:** `include/app/sinks.hpp`
- **Description:** A basic helper class managing binary file outputs.

Methods:

- `explicit FileSink(const std::string& path)`
 - Opens a binary write stream to the target path.
- `void write(const uint8_t* data, size_t len)`
 - Appends the specified bytes to the file output buffer.
- `void flush()`
 - Flushes outstanding buffers to disk.

5. `moq::app::Scheduler`

- **File Path:** `include/app/scheduler.hpp`
- **Description:** Determines stream routing for prioritizing different types of media chunks.

Methods:

- `explicit Scheduler(uint32_t num_streams)`
 - Creates a scheduler initialized for the specified stream allocation size.
- `uint32_t pick_stream(uint32_t nalu_id) const`
 - Selects a target stream index based on NALU properties (e.g., priority-based mapping or round-robin fallback).

Network & QUIC Transport

1. `moq::transport::QuicConfig`

- **File Path:** `include/transport/quic_transport.hpp`
- **Description:** Simple structure encapsulating initialization configuration parameters.

```
struct QuicConfig {
    std::string host;           ///< Target IP address or hostname.
    uint16_t port{0};          ///< Target UDP port.
    uint32_t num_streams{1};    ///< Pre-allocated bidirectional stream count.
    std::string cert_file;      ///< Path to server TLS certificate file (server mode).
    std::string key_file;       ///< Path to server TLS private key file (server mode).
    uint16_t min_mtu{0};        ///< Custom minimum Path MTU override (optional).
    uint16_t max_mtu{0};        ///< Custom maximum Path MTU override (optional).
};
```

2. `moq::transport::MsQuicTransport`

- **File Path:** `include/transport/msquic_transport.hpp`
- **Description:** Wraps Microsoft's MsQuic C library handles, credentials, configurations, and callbacks inside a thread-safe C++ object.

Shared APIs:

- `MsQuicTransport()`
 - Opens the MsQuic API function table and registers registration hooks.
- `void run()`

- Blocks the calling thread (listener loop) until the transport is shut down.
- `void set_recv_callback(PacketRecvCb cb)`
 - Registers a callback matching `std::function<void(const PacketHeader&, const uint8_t*, size_t)>` to handle fully parsed application-layer packets.

Client APIs:

- `bool start_client(const QuicConfig& cfg)`
 - Establishes a connection to the configured host and port. Pre-allocates and starts the requested number of streams.
- `bool wait_connected()`
 - Blocks until the client TLS handshake is complete.
- `bool wait_stream_ready()`
 - Blocks until client streams are initialized and ready to transmit.
- `bool set_stream_priority(uint32_t stream_index, uint16_t priority)`
 - Invokes MSQuic stream priority parameter overrides (e.g., `0xFFFF` for critical priority, `0x0001` for standard priority).
- `bool send(uint32_t stream_index, const moq::PacketHeader& hdr, const uint8_t* payload)`
 - Serializes header and payload into a `SendContext` and schedules an asynchronous `StreamSend` operation.
- `void set_send_expected(uint32_t n)`
 - Sets the number of packets expected to be successfully acknowledged.
- `bool wait_all_sends_done(uint32_t timeout_ms = 5000)`
 - Blocks until the configured number of expected sends has triggered `SEND_COMPLETE` callbacks.
- `void graceful_shutdown()`
 - Initiates stream shutdowns across all open client streams.
- `bool wait_stream_shutdown(uint32_t timeout_ms)`
 - Blocks until all streams complete their shutdown negotiations.

Server APIs:

- `bool start_server(const QuicConfig& cfg)`
 - Starts a server listener on the designated port and registers credentials (cert and private key).

Simple Integration Walkthrough

Below is a conceptual example illustrating how to parse a raw H.265 file, chunk it, send it over multiple prioritized streams using `MsQuicTransport`, and reassemble it on the receiver side:

Client Sender Logic

```

#include "app/h265_parser.hpp"
#include "app/packetizer.hpp"
#include "transport/msquic_transport.hpp"
#include <iostream>

int main() {
    // 1. Parse H.265 NALUs
    moq::app::H265FileParser parser("test_input.h265");
    if (!parser.open()) return -1;
    const auto& nalus = parser.get_nalus();

    // 2. Configure & Start Transport
    moq::transport::QuicConfig cfg;
    cfg.host = "127.0.0.1";
    cfg.port = 4433;
    cfg.num_streams = 2; // Stream 0 (I-frames), Stream 1 (P-frames)

    moq::transport::MsQuicTransport quic;
    if (!quic.start_client(cfg) || !quic.wait_connected() || !quic.wait_stream_ready()) {
        return -1;
    }

    // Assign priorities: Stream 0 is high priority, Stream 1 is low
    quic.set_stream_priority(0, 0xFFFF);
    quic.set_stream_priority(1, 0x0001);

    // 3. Initialize Packetizer
    moq::app::DefaultPacketizer packetizer(1200); // 1200-byte payloads

    // 4. Send Packetized Fragments
    for (size_t i = 0; i < nalus.size(); ++i) {
        // High priority stream for metadata/I-frames, standard stream for P-frames
        uint32_t stream_idx = nalus[i].is_important() ? 0 : 1;

        auto fragments = packetizer.packetize(nalus[i], static_cast<uint32_t>(i));
        for (const auto& fragment : fragments) {
            quic.send(stream_idx, fragment.hdr, fragment.payload.data());
        }
    }

    // 5. Close connection and wait for stream shutdown ACK
    quic.graceful_shutdown();
    quic.wait_stream_shutdown(5000);
    return 0;
}

```

Server Receiver Logic

```

#include "app/reassembler.hpp"
#include "app/sinks.hpp"
#include "transport/msquic_transport.hpp"
#include <iostream>

int main() {
    // 1. Setup Sink and Reassembler
    moq::app::FileSink sink("received_output.h265");
    moq::app::Reassembler reasm([&](const uint8_t* data, size_t len) {
        sink.write(data, len); // Flushes contiguous, in-order NALUs to disk
    });

    // 2. Setup Transport
    moq::transport::QuicConfig cfg;
    cfg.host = "0.0.0.0";
    cfg.port = 4433;
    cfg.cert_file = "server.crt";
    cfg.key_file = "server.key";

    moq::transport::MsQuicTransport quic;
    quic.set_rcv_callback([&](const moq::PacketHeader& hdr, const uint8_t* payload, size_t len) {
        reasm.on_packet(hdr, payload, len); // Feed fragments into reassembler
    });

    if (!quic.start_server(cfg)) return -1;

    // 3. Block and listen
    quic.run();
    sink.flush();
    return 0;
}

```